

Dual-Issue Superscalar RISC-V Processor

Design & Implementation in TL-Verilog

Base platform	WARP-V RISC-V Core
Language	TL-Verilog
Environment	Makerchip IDE
ISA	RV32I + Zicsr counters
Branch predictor	Two-bit saturating
Issue width	2 (in-order)

Contents

1	Introduction	3
1.1	Project Objective	3
1.2	Scalar and Superscalar Execution	3
1.3	Design Principle	3
2	The Baseline WARP-V Core	3
2.1	TL-Verilog Features	3
2.2	Generated Configuration	4
2.3	Pipeline Organisation and the Instruction Scope	4
2.4	Instruction Fetch and the Program Counter	4
2.5	Instruction Decode	4
2.6	Register Read and the Bypass Network	5
2.7	The Pending Bit and Replay	6
2.8	The Two-Bit Saturating Branch Predictor	6
2.8.1	The state machine	6
2.8.2	Prediction and update in the baseline	7
2.8.3	The two redirects a predictor implies	7
2.9	Speculation Control: the Good-Path Mask	8
2.10	Execute Stage	9
2.11	Load Handling and Second Issue	10
2.12	Data Memory	10
2.13	Control and Status Registers	10
2.14	Commit and the Built-in Test Program	10
3	Dual-Issue Architecture	11
3.1	Replicating the Transaction, Not the Pipeline	11
3.2	Fetch Groups and Slot Ordering	11
3.3	Shared versus Replicated Logic	11
3.4	Overall Structure	11
4	Implementation	12
4.1	Replication of the Instruction Scope	12
4.2	Per-Slot Program Counter and Fetch	12
4.3	Instruction-Memory Extension	13
4.4	Per-Slot Target and Result Computation	13
4.5	The Shared Next-PC Arbiter	13
4.6	The Asymmetric Good-Path Mask	14
4.7	Redirect Summaries and the Slot-1 Kill	14
4.8	The Six-Way Bypass Network	15
4.9	Register File with Two Write Ports	16
4.10	Shared Single-Port Data Memory	16
4.11	Superscalar-Aware CSRs	17
4.12	Load-Return	17
4.13	Porting the Two-Bit Saturating Branch Predictor	18
4.13.1	(1) Hoisting the counter and applying two updates per cycle	18
4.13.2	(2) Per-slot prediction from the shared counter	19
4.13.3	(3) Splitting the predicted-taken redirect	19
4.13.4	(4) Direction-aware mispredict recovery	20
4.13.5	Summary of the port	20

5	The Hazards of Dual Issue	21
5.1	Same-Cycle Conflicts	21
5.2	Read-After-Write Between the Slots	21
5.3	Write-After-Write Between the Slots	21
5.4	Contention on the Memory Port	22
5.5	Contention on the CSR Port	22
5.6	The Replay Resolution	22
5.7	Control Hazards Between the Slots	22
6	Some Extra Analysis	23
6.1	Replay Timing	23
6.2	Cross-Slot Redirect Races	23
6.3	Predictor Behaviour Under Dual Issue	23
A	Summary of Changes	25

1 Introduction

1.1 Project Objective

This report documents the design and implementation of a **two-wide, in-order Superscalar RISC-V processor** in TL-Verilog. The processor is derived from a single-issue core produced by the WARP-V core generator, which offers no superscalar or multi-issue option of any kind; the dual-issue capability documented here is therefore an original extension layered on top of a generated single-issue baseline. A second execution path is realised through TL-Verilog’s *hierarchy and replication* features—by making the per-instruction scope hold two instructions instead of one—and not by physically duplicating the pipeline. The report first describes the operation of the unmodified baseline core in full (Section 2), then presents the dual-issue architecture (Section 3), the implementation of every mechanism (Section 4), the new hazards created by issuing two instructions per cycle together with their resolutions (Section 5), and further analysis (Section 6).

1.2 Scalar and Superscalar Execution

A scalar, single-issue processor fetches, decodes, and completes at most one instruction per clock cycle, bounding its throughput at one instruction per cycle ($IPC = 1$). A **superscalar** processor widens the machine so that multiple instructions occupy the same pipeline stage simultaneously, raising the achievable IPC above one. The processor described here is **two-wide**: in every cycle it can fetch, decode, and retire up to two instructions. It is also strictly **in-order**—instructions issue and retire in program order with no dynamic reordering.

1.3 Design Principle

The per-instruction scope `/instr` is replicated into two slots—slot 0 (the *older* instruction in program order) and slot 1 (the *younger*)—that travel down one shared pipeline together. Every genuinely shared structure (program counter, register file, data memory, CSR counters, branch-predictor state) is kept as a single copy hoisted above the two slots. On top of this structure, exactly the logic is added which guarantees that **slot 0 is never harmed by anything slot 1 does, and slot 1 is always killed by anything slot 0 does that changes control flow**.

2 The Baseline WARP-V Core

2.1 TL-Verilog Features

Two properties of TL-Verilog underpin both the baseline and its extension:

- **Automatic pipelining.** A signal `$sig` assigned in a stage `@n` is registered through the pipeline automatically. The reference `>>1$sig` reads its value from one cycle earlier and `>>3$sig` from three cycles earlier; the required flip-flops are inferred. No pipeline registers are ever declared by hand, which is what allows the forwarding network to be expressed in a handful of readable lines.
- **Hierarchy and replication.** A scope such as `/instr` can be declared as an array, `/instr[1:0]`, whereupon the entire body of logic beneath it is instantiated once per index. The elaboration-time constant `#instr` identifies each instance, so a single description can specialise its copies. This single feature is what converts one issue slot into two without a duplicated pipeline.

2.2 Generated Configuration

The baseline was generated with a realistic mid-depth pipeline and a two-bit saturating branch predictor.

Setting	Value	Consequence for the design
Pipeline depth	Fetch @1; decode, register read, branch predict @2; execute @3; result @4; register write and memory @5	A forwarding reach of three cycles, which the dual-issue network later extends.
Branch prediction	Two-bit saturating counter	Introduces an early, stage-2 <i>predicted-taken</i> redirect in addition to the stage-3 mispredict redirect, and adds a piece of shared predictor state that must be hoisted alongside the register file.
Hazard bubbles	<code>EXTRA_NON_PIPELINED_BUBBLE = 1</code> , <code>EXTRA_TRAP_BUBBLE = 1</code> , others 0	Fixes the redirect-timing alignments (>>3 for traps) preserved verbatim by the extension.
ISA	RV32I + Zicsr counters	Base integer instructions plus the <code>cycle</code> , <code>time</code> , and <code>instret</code> counters.

Because the generator offers no superscalar option, no alternative configuration could have shortened the work; this configuration simply provides the friendliest baseline to restructure.

2.3 Pipeline Organisation and the Instruction Scope

The core contains one pipeline, `|fetch`. Every signal belonging to a specific instruction lives in the scope `/instr` within it; WARP-V models each in-flight instruction as a *transaction* flowing through that scope, with exactly one transaction per stage per cycle in the baseline. Decode validity simply follows fetch:

```
$valid_decode = $fetch; // Always decode if we fetch.
```

2.4 Instruction Fetch and the Program Counter

Each cycle the core forms the next fetch address, reads the corresponding 32-bit word from the on-chip instruction array, and begins decoding it:

```
@1
? $fetch
$fetch_word[31:0] = *instrs[/instr$Pc[5:2]];
```

A single register `$Pc` holds the program counter, updated every cycle from a `$next_pc` value selected by the redirect logic of Section 2.9. The baseline’s sequential default is the current PC plus 4. This is the first quantity the dual-issue extension must change.

2.5 Instruction Decode

At the decode stage the raw word is sliced into its RISC-V fields, the instruction is classified, and immediates are extracted per format. The two source-register indices are produced inside a small

replicated sub-scope `/src[2:1]`, one instance per operand:

```
/src[2:1]
  // Reg valid for this source, based on instruction type.
  $is_reg =

    (/instr$is_r_type || /instr$is_r4_type || (/instr$is_i_type && (#src == 1)) || /
    ↪ instr$is_r2_type || /instr$is_s_type || /instr$is_b_type);
  $reg[4:0] = (#src == 1) ? /instr$raw_rs1[4:0] : /instr$raw_rs2[4:0];
```

The destination register follows directly from the decoded fields, with `x0` excluded since writes to it are architecturally discarded:

```
$dest_reg[4:0] = $raw_rd[4:0]; // int only
$dest_reg_valid = $valid_decode && ! $is_s_type && ! $is_b_type &&
                  | $dest_reg; // r0 not valid. Asserts for int dest only (see /fpu/
    ↪ $dest_reg_valid for FP).
```

CSR instructions are recognised here as well, together with a trap condition for unsupported CSR addresses:

```
// CSR decode.
$is_csr_write = $is_csrrw_instr || $is_csrrwi_instr;
$is_csr_set   = $is_csrrs_instr || $is_csrrsi_instr;
$is_csr_clear = $is_csrrc_instr || $is_csrrci_instr;
$is_csr_instr = $is_csr_write ||
                 $is_csr_set   ||
                 $is_csr_clear;
$valid_csr = 1'b0 || $is_csr_cycle || $is_csr_cycleh || $is_csr_time || $is_csr_timeh ||
    ↪ $is_csr_instret || $is_csr_instreth;
```

These decode results like load/store classification, branch classification, source and destination indices are precisely the signals the dual-issue hazard detectors compare between slots.

2.6 Register Read and the Bypass Network

The register file holds the writable registers `x1–x31`. Reading a register that an in-flight instruction is about to write would return a stale value, so the value is *forwarded*: if a source index matches the destination of the instruction one, two, or three cycles ahead, the in-flight result is used instead of the register-file copy. The baseline network therefore offers **three** bypass options:

```
$bypass_avail1 = >>1$commit_dest_reg && (/instr$GoodPathMask[1] || /instr>>1$commit_second_issue)
    ↪ ;
$bypass_avail2 = >>2$commit_dest_reg && (/instr$GoodPathMask[2] || /instr>>2$commit_second_issue)
    ↪ ;
$bypass_avail3 = >>3$commit_dest_reg && (/instr$GoodPathMask[3] || /instr>>3$commit_second_issue)
    ↪ ;
/src[2:1]
  $is_reg_condition = $is_reg && /instr$valid_decode; // Note: $is_reg can be set for RISC-V
    ↪ sr0.
  ?$is_reg_condition
    $rf_value[31:0] =
      /instr/regs[$reg]>>3$value;
    {$reg_value[31:0], $pending} =
```

```

    ($reg == 5'b0) ? {32'b0, 1'b0} : // Read r0 as 0 (not pending).
    // Bypass stages. Both register and pending are bypassed.
    (/instr$bypass_avail1 && (/instr>>1$wr_reg == $reg)) ? {/instr>>1$rslt, /instr>>1
    ↪ $reg_wr_pending} :
    (/instr$bypass_avail2 && (/instr>>2$wr_reg == $reg)) ? {/instr>>2$rslt, /instr>>2
    ↪ $reg_wr_pending} :
    (/instr$bypass_avail3 && (/instr>>3$wr_reg == $reg)) ? {/instr>>3$rslt, /instr>>3
    ↪ $reg_wr_pending} :
    {$rf_value, /instr/regs[$reg]>>3$pending};
    // Replay if source register is pending.

```

Both the register value *and* its pending bit (next subsection) are forwarded, and the availability qualifiers (`$bypass_avail1..3`) require the producer to have committed its write on the good path. The three-candidate structure is a direct consequence of the pipeline depth, and is the structure the dual-issue extension widens.

2.7 The Pending Bit and Replay

Alongside each register value the core keeps a `$pending` bit meaning “this value is not yet available”—its producer, typically a load, is still in flight:

```

/regs[31:1]
<<1$pending = ! /instr$reset && (((#regs == /instr$wr_reg) && /instr$commit_dest_reg) ? /
    ↪ instr$reg_wr_pending : $pending);

```

Any instruction that reads a pending source must **replay**: it aborts itself and is re-fetched, retrying until the value has landed. An instruction that *will* produce a late value advertises this when it commits:

```

$non_pipelined = $div_mul ;
$would_second_issue = ($ld && 1'b1) || $non_pipelined;
// Pending value to write to dest reg, 1 for instructions that will second-issue.
$reg_wr_pending = $would_second_issue && ! $second_issue;

```

Replay is a general recovery mechanism already present in the baseline core, and Section 5 shows that every hazard created by dual issue is resolved by re-using this same mechanism.

2.8 The Two-Bit Saturating Branch Predictor

Branches are resolved only at the execute stage, three cycles after fetch. Without prediction the fetch unit would have to guess, and every wrong guess costs a pipeline flush. The baseline core therefore contains a **two-bit saturating counter** predictor, the classic bimodal scheme.

The state machine

A single two-bit register, `$BranchState`, encodes a confidence level rather than a bare last-outcome bit:

State	Meaning	Prediction	Next state (taken / not-taken)
2'b00	Strongly not-taken	Not taken	2'b01 / 2'b00 (saturates)
2'b01	Weakly not-taken	Not taken	2'b10 / 2'b00
2'b10	Weakly taken	Taken	2'b11 / 2'b01
2'b11	Strongly taken	Taken	2'b11 (saturates) / 2'b10

The prediction is simply the most significant bit of the counter. The value of the second bit is *hysteresis*: a branch that has been taken many times sits in 2'b11, and a single not-taken outcome moves it only to 2'b10, where it still predicts taken. A one-bit predictor would mispredict twice per loop—once on the final exit and once on the first iteration of the next entry—whereas the two-bit counter absorbs the exit as a single mispredict and keeps predicting taken. The counter is initialised to 2'b01 (weakly not-taken) at reset.

Prediction and update in the baseline

The complete predictor is these few lines:

```
// Instantiate the branch predictor.

@2
? $branch
    $pred_taken = >>2 $BranchState[1];

@3
$branch_or_reset = ($branch && $commit) || $reset;
? $branch_or_reset
    $BranchState[1:0] <=
        $reset ? 2'b01 :
        $taken ? ($BranchState == 2'b11 ? $RETAIN : $BranchState + 2'b1) :
        ($BranchState == 2'b00 ? $RETAIN : $BranchState - 2'b1);
```

Three properties of this code matter for the port:

- **It is a single global counter, not a table.** There is no indexing by PC; every branch in the program shares one counter. This keeps the predictor small and is the scheme the extension must reproduce.
- **Prediction is read with a >>2 alignment.** The counter is written at stage 3 and read at stage 2. The two-transaction alignment guarantees the value read is already settled, so no combinational loop is formed between the update and the prediction.
- **The update is gated on commit.** `$branch_or_reset` asserts only when a branch actually retires on the good path (`$branch && $commit`), so wrong-path branches never pollute the predictor. `$RETAIN` is the TL-Verilog keyword for “hold the current value”, which implements saturation.

The two redirects a predictor implies

With a predictor present, a branch can steer the program counter at *two* different points, and the baseline defines a separate redirect condition for each.

(1) Predicted-taken, at stage 2. As soon as decode identifies a branch and the counter predicts taken, fetch is redirected to the branch target speculatively:


```
@2
$pred_taken_branch = $pred_taken && $branch;
```

```
$pred_taken_branch_cond = (>>1$pred_taken_branch && !(1'b0 || >>1$no_fetch || >>1$second_issue)
    ↪ && $GoodPathMask[1]);
```

This condition is consumed at $\gg 1$, so it kills only the single instruction already fetched behind the branch; its good-path mask term is correspondingly 4'b1110, clearing one bit.

(2) Mispredict, at stage 3. When the branch actually resolves, the outcome is compared against the prediction:

```
$mispred_branch = $branch && ! ($conditional_branch && ($taken == $pred_taken));
? $valid_decode_branch
  $branch_redir_pc[31:0] =
    // If fallthrough predictor, branch mispred always redirects taken, otherwise PC+1 for not-
    ↪ taken.
    (! $taken) ? $pc_inc :
    $branch_target;
```

The recovery target now depends on the direction of the error. A branch predicted not-taken but resolved taken must redirect to `$branch_target`; a branch predicted taken but resolved not-taken must redirect to `$pc_inc`, the fall-through address. This second case does not exist in a fallthrough-only predictor, where a mispredict always redirects to the target.

Why this matters for dual issue

The predictor adds one shared state element and one extra redirect condition. Both have direct consequences for the two-wide core: the state must be hoisted so a single counter serves both slots, and the new stage-2 redirect must be split into slot-0 and slot-1 forms and given a good-path mask term, exactly as every other redirect condition already has been. Section 4.13 covers the port.

2.9 Speculation Control: the Good-Path Mask

Because the pipeline fetches speculatively, the core must track which in-flight instructions are legitimately on the correct execution path. WARP-V maintains a shifting bit-vector, `$GoodPathMask`—one bit per cycle in flight, a 1 meaning “good path.” Each cycle a set of *redirect conditions* is evaluated (predicted-taken branch, mispredicted branch, jump, trap, replay, load return); when one fires, the PC is steered to the corresponding target and the mask bits of the newly-invalidated younger instructions are cleared:

```
$next_good_path_mask[3+1:0] =
  // Shift up and mask w/ redirect conditions.
  {$GoodPathMask[3:0]
    & ($no_fetch_cond ? 4'b1110 : 4'b1111) & ($second_issue_cond ? 4'b1110 : 4'b1111) & (
    ↪ $pred_taken_branch_cond ? 4'b1110 : 4'b1111) & ($replay_cond ? 4'b1100 : 4'b1111) & (
    ↪ $jump_cond ? 4'b1100 : 4'b1111) & ($mispred_branch_cond ? 4'b1100 : 4'b1111) & (
    ↪ $indirect_jump_cond ? 4'b1100 : 4'b1111) & ($non_pipelined_cond ? 4'b1000 : 4'b1111) & (
    ↪ $aborting_trap_cond ? 4'b0000 : 4'b1111) & ($non_aborting_trap_cond ? 4'b1000 : 4'b1111),
    1'b1}; // Shift in 1'b1 (fetch-valid).
```

```
$GoodPathMask[3+1:0] <=
  <<1$reset ? 5'b0 : // All bad-path (through self) on reset (next mask based on next reset).
  $next_good_path_mask;
```

The priority-encoded target selection follows; a condition resolved at a *later* stage belongs to an *older* instruction whose decision is more final, and therefore outranks earlier-stage conditions:

```
{ $next_pc[31:0], $next_no_fetch } =
  $reset ? { 31'b0, 2'b0 } :
  $non_aborting_trap_cond ? { >>3$trap_target, 1'b0 } : $aborting_trap_cond ? { >>3$trap_target, 1'
  ↪ b0 } : $non_pipelined_cond ? { >>3$pc_inc, 1'b1 } : $indirect_jump_cond ? { >>2
  ↪ $indirect_jump_target, 1'b0 } : $mispred_branch_cond ? { >>2$branch_redir_pc, 1'b0 } :
  ↪ $jump_cond ? { >>2$jump_target, 1'b0 } : $replay_cond ? { >>1$Pc, 1'b0 } :
  ↪ $pred_taken_branch_cond ? { >>1$branch_target, 1'b0 } : $second_issue_cond ? { >>0$Pc, 1'b0 }
  ↪ : $no_fetch_cond ? { >>0$Pc, 1'b1 } :
    ({ $pc_inc, 1'b0 });
// Then as state.
$Pc[31:0] <= $next_pc;
```

Note where the predicted-taken condition sits: below `$replay_cond`, because a replay is aborting and must win, and above `$second_issue_cond`. An instruction may **commit** its results only if its mask bit is still set and it did not itself abort. This mask machinery, and its priority ordering, is what the dual-issue design re-derives but for two instructions.

2.10 Execute Stage

The execute stage produces results for every instruction class. Branch direction is resolved by comparing the two source operands according to the branch type:

```
?$valid_exe
  $equal = /src[1]$reg_value == /src[2]$reg_value;
?$branch
  $taken =
    $is_j_type ||
    ($is_beq_instr && $equal) ||
    ($is_bne_instr && ! $equal) ||
    (($is_blt_instr || $is_bltu_instr || $is_bge_instr || $is_bgeu_instr) &&
```

Memory instructions form their effective address from the first source operand and the immediate, and stores align their data and byte-enable mask to the low address bits so that sub-word writes touch only the intended bytes:

```
?$st_cond
  // Provide a value to store, naturally-aligned to memory, that will work regardless of the
  ↪ lower $addr bits.
  $st_reg_value[31:0] =
    /src[2]$reg_value;

  $st_value[31:0] =
    $ld_st_word ? $st_reg_value : // word
    $ld_st_half ? { 2{$st_reg_value[15:0]} } : // half
```

Every computation in this stage that references the program counter, such as branch targets, jump

targets, AUIPC, and link addresses become per-slot in the dual-issue design, since the two slots reside at different addresses.

2.11 Load Handling and Second Issue

Loads cannot deliver their value on the first pass because memory has latency. A load therefore commits immediately, writing a placeholder and setting `$pending` on its destination; several cycles later, when the data returns, the load **second-issues** as it re-enters the pipeline, clobbers whatever instruction occupies the slot it lands in, and performs the real register write, clearing the pending bit:

```
$second_issue_ld = ! $reset && /instr>>5$valid_ld && 1'b1;
```

Dependent instructions replay in the interval and eventually observe the correct value. The returned data is formatted with byte selection and sign extension before the final write:

```
{ $ld_rslt[31:0], $ld_mask[3:0] } =
  $ld_st_word ? { $ld_data, 4'b1111 } :
  $ld_st_half ? { {16{$sign_bit}}, $addr[1] ? { $ld_data[31:16], 4'b1100 } :
    { $ld_data[15:0], 4'b0011 } } :
    { {24{$sign_bit}}, ($addr[1:0] == 2'b00) ? { $ld_data[7:0], 4'b0001 } :
```

This mechanism is elegant in a single-issue core but forces a modification in dual-issue: a returning load is an *old* instruction and must never occupy the younger slot beside a newly-fetched instruction, or program order would invert. The resolution appears in Section 4.12.

2.12 Data Memory

The data memory is a byte-addressable array organised as four byte-banks, so byte and half-word stores update only the affected banks. It provides a **single** load/store port—one memory operation per cycle. In the baseline this memory resides *inside* `/instr`; that placement is the reason it must be hoisted before the scope is replicated.

2.13 Control and Status Registers

The core implements the RISC-V counter CSRs—`cycle` (elapsed clock cycles), `time` (a divided real-time counter), and `instret` (retired instructions)—each as a 64-bit quantity split across a low/high register pair, read by CSR instructions. In the baseline, `instret` advances by one per committed instruction. Register-file writes themselves are performed at the final stage:

```
always @ (posedge clk) begin
  if (/instr$commit_dest_reg)
    /regs[$wr_reg]<<0$$^value[31:0] <= /instr$rslt;
end
```

2.14 Commit and the Built-in Test Program

An instruction's commit decision combines good-path status with the absence of an abort.

3 Dual-Issue Architecture

3.1 Replicating the Transaction, Not the Pipeline

The obvious route to a two-wide machine, building a second copy of the pipeline duplicates hardware that must physically exist only once, such as the register file and the data memory. The architecture instead exploits the fact that all per-instruction behaviour already resides in one scope. TL-Verilog permits that scope to become an array of two, `/instr[1:0]`, whereupon the toolchain builds the entire body—decode, operand read, execute, result, commit—**twice** from a single description. The logic is written once; two issue slots result. Everything genuinely shared is deliberately kept *outside* the replicated scope as a single instance.

3.2 Fetch Groups and Slot Ordering

Each cycle fetches a **fetch group** of two consecutive instructions that traverse the pipeline together:

- **Slot 0** (`/instr[0]`)—the *older* instruction, fetched at `$Pc`;
- **Slot 1** (`/instr[1]`)—the *younger* instruction, fetched at `$Pc + 4`.

With two instructions consumed per cycle, the sequential stride becomes **eight bytes**. A control-flow redirect may target any 4-byte-aligned address; the next group is simply the target and the following word, so no fetch-alignment or rotation hardware is required.

3.3 Shared versus Replicated Logic

Every element of the core was sorted into two categories: logic describing a single instruction (safe to replicate) and hardware representing shared machine state (necessarily singular). The latter is **hoisted** out of `/instr` into the enclosing `|fetch` scope so that exactly one copy exists.

Replicated per slot (inside <code>/instr[1:0]</code>)	Hoisted and shared (in <code> fetch</code>)
Decode and field extraction	Program counter and next-PC arbitration
Immediate generation; illegal-instruction check	Good-path mask and redirect priority
Operand read and per-slot forwarding muxes	Architectural register file (two write ports)
ALU, execute, result selection	Data memory (one load/store port)
Per-slot commit / abort decision	CSR counter state
Load-address and store-data formation	Branch-predictor state (<code>\$BranchState</code>)
Branch prediction read and <code>\$pred_taken_branch</code>	Forwarding-availability signals
CSR address decode and write value	Test-bench pass/fail evaluation

3.4 Overall Structure

The finished design has the following structure – a shared spine directly in `|fetch`, the two slots inside the replicated `/instr[1:0]`, and the shared state structures alongside, each a single instance:

```
|fetch                                // one shared pipeline
  @1 program counter, redirect arbitration, good-path mask
  @2 slot-0 redirect summary, forwarding availability (shared)
  @3 slot-0 redirect summary (final), test pass/fail,
    two-bit branch-predictor state (ONE counter, two update steps)
/instr[1:0]                          // TWO slots built from ONE description
  @1 per-slot PC and fetch; route returning loads to slot 0
  @2 decode, operand read + 6-way forwarding, hazard detect, replay,
    branch prediction read + predicted-taken redirect
```

```

    @3 execute, address/data formation, commit decision, slot-1 kill
    @4 result selection
/regs[31:1] @5           // shared register file, TWO write ports
/bank[3:0]/mem @5        // shared data memory, ONE port
(CSR counter state)      // shared; software access from slot 0

```

4 Implementation

4.1 Replication of the Instruction Scope

The single change that makes the processor superscalar is the conversion of the instruction scope into an array. Where the baseline declares `/instr`, the dual-issue core declares `/instr[1:0]`, and the entire body beneath it is instantiated twice:

```

/instr[1:0]
@0
    $reset = |fetch$reset;
@1
    $fetch = |fetch$fetch;
    // Per-slot PC: slot 0 fetches at the group PC, slot 1 at group PC + 4.
    $pc[31:0] = |fetch$Pc + (#instr << 2);
    $pc_inc[31:0] = $pc + 32'd4;

```

From this point every signal inside the slot exists in two copies; wherever the copies must differ, the elaboration constant `#instr` (0 for slot 0, 1 for slot 1) is checked. Its first use is already visible in the per-slot program counter, described next.

4.2 Per-Slot Program Counter and Fetch

Both slots share the group program counter, but each derives its own address from it: slot 0 uses the group PC directly, and slot 1 adds four. The expression `#instr << 2` evaluates to “instance number times four”:

```

$fetch = |fetch$fetch;
// Per-slot PC: slot 0 fetches at the group PC, slot 1 at group PC + 4.
$pc[31:0] = |fetch$Pc + (#instr << 2);
$pc_inc[31:0] = $pc + 32'd4;
$no_fetch = 1'b0;
// A returning load clobbers a slot. All load returns are steered into SLOT 0
// (regardless of which slot originally issued the load), and the whole current
// group is refetched ($second_issue_cond redirects to >>0$Pc). This keeps the
// register-file write ordering trivially correct: a returning load can never
// co-commit with a younger slot-1 instruction of the same cycle.
$second_issue_ld = (#instr == 0) && ! $reset && (|fetch/instr[0]>>5$valid_ld || |fetch/instr
    ↪ [1]>>5$valid_ld);
// Which slot the returning load originally issued from (for pulling its signals).
$ld_ret_s1 = |fetch/instr[1]>>5$valid_ld;
$second_issue = ($second_issue_ld);

```

The `$second_issue_ld` and `$ld_ret_s1` assignments concern returning loads and is explained in Section 4.12.

4.3 Instruction-Memory Extension

Fetching two instructions per cycle implies that, near the end of the program, slot 1 reads one entry *past* the final real instruction. To guarantee that this can never yield an undefined value, the instruction array was widened and padded with NOPs (`ADDI x0,x0,0`) to a power-of-two depth, so the index taken from the PC can never leave the array.

4.4 Per-Slot Target and Result Computation

Every execute-stage computation that referenced the program counter was moved from the group PC to the slot's own `$pc`:

```
$branch_target[31:0] = $pc[31:0] + $raw_b_imm[31:0]; // per-slot branch target
$jump_target[31:0]   = $pc[31:0] + $raw_j_imm[31:0]; // per-slot jump target
$auipc_rslt[31:0]    = $pc + $raw_u_imm;           // per-slot AUIPC
$jal_rslt[31:0]      = $pc + 4;                     // per-slot link address
$jalr_rslt[31:0]     = $pc + 4;
```

Left on the group PC, slot 1 would compute every branch and jump target four bytes low. This change is what allows both instructions of a group to be architecturally correct simultaneously. It applies to the predicted-taken redirect as well: each slot supplies *its own* `$branch_target` to the next-PC arbiter.

4.5 The Shared Next-PC Arbiter

A redirect can now originate in either slot, and the two cases require different treatment. Every redirect condition was split into a slot-0 form and a slot-1 form, each qualified by the group's good-path bit; the slot-1 forms carry an additional qualifier, (`! >>N$slot0_redir_e3`), expressing “if slot 0 of this group has already redirected, slot 1 is on the wrong path and must not steer the PC.” A single priority multiplexer then selects the target:

```
{ $next_pc[31:0], $next_no_fetch } =
  $reset ? { 32'b0, 1'b0 } :
  $non_aborting_trap_cond_0 ? { /instr[0]>>3$trap_target, 1'b0 } :
  $aborting_trap_cond_0     ? { /instr[0]>>3$trap_target, 1'b0 } :
  $non_pipelined_cond_0    ? { /instr[0]>>3$pc_inc, 1'b1 } :
  $non_aborting_trap_cond_1 ? { /instr[1]>>3$trap_target, 1'b0 } :
  $aborting_trap_cond_1     ? { /instr[1]>>3$trap_target, 1'b0 } :
  $non_pipelined_cond_1    ? { /instr[1]>>3$pc_inc, 1'b1 } :
  $indirect_jump_cond_0    ? { /instr[0]>>2$indirect_jump_target, 1'b0 } :
  $mispred_branch_cond_0   ? { /instr[0]>>2$branch_redir_pc, 1'b0 } :
  $jump_cond_0             ? { /instr[0]>>2$jump_target, 1'b0 } :
  $indirect_jump_cond_1    ? { /instr[1]>>2$indirect_jump_target, 1'b0 } :
  $mispred_branch_cond_1   ? { /instr[1]>>2$branch_redir_pc, 1'b0 } :
  $jump_cond_1             ? { /instr[1]>>2$jump_target, 1'b0 } :
  $replay_cond_0           ? { /instr[0]>>1$pc, 1'b0 } : // Refetch the group from slot 0.
  $replay_cond_1           ? { /instr[1]>>1$pc, 1'b0 } : // Refetch from slot 1's address: it
    ↳ becomes slot 0 of the next group.
  $pred_taken_branch_cond_0 ? { /instr[0]>>1$branch_target, 1'b0 } : // Speculatively follow
    ↳ slot 0's predicted-taken branch.
  $pred_taken_branch_cond_1 ? { /instr[1]>>1$branch_target, 1'b0 } : // Same for slot 1 (only
    ↳ if slot 0 did not redirect).
  $second_issue_cond        ? { >>0$Pc, 1'b0 } :
  $no_fetch_cond            ? { >>0$Pc, 1'b1 } :
    { $Pc + 32'd8, 1'b0 }; // Default: sequential fetch of the next group of two.
  // Then as state.
```

The ordering is based in our rule: **a later pipeline stage wins** (older instruction, more final decision),

and **within a stage, slot 0 precedes slot 1**. The default case is $\$Pc + 32'd8$, the two-instruction sequential stride. The two replay cases target the individual slot's own PC—the property on which the hazard resolution of Section 5 rests. The two predicted-taken cases sit immediately below the replays, preserving the baseline's relative priority (a replay is aborting and must win over a speculative prediction) and above $\$second_issue_cond$.

4.6 The Asymmetric Good-Path Mask

Each condition clears mask bits according to how many younger instructions it invalidates. Slot-0 conditions retain the original single-issue patterns; slot-1 *aborting* conditions clear **one bit fewer**, so the group's own bit survives and slot 0 still commits:

```
$next_good_path_mask[3+1:0] =
  // Shift up and mask w/ redirect conditions.
  {$GoodPathMask[3:0]
    // Slot-0-triggered conditions keep original (single-issue) mask semantics:
    & ($no_fetch_cond      ? 4'b1110 : 4'b1111)
    & ($second_issue_cond  ? 4'b1110 : 4'b1111)
    & ($replay_cond_0     ? 4'b1100 : 4'b1111)
    & ($jump_cond_0       ? 4'b1100 : 4'b1111)
    & ($mispred_branch_cond_0 ? 4'b1100 : 4'b1111)
    & ($indirect_jump_cond_0 ? 4'b1100 : 4'b1111)
    & ($non_pipelined_cond_0 ? 4'b1000 : 4'b1111)
    & ($aborting_trap_cond_0 ? 4'b0000 : 4'b1111)
    & ($non_aborting_trap_cond_0 ? 4'b1000 : 4'b1111)
    & ($pred_taken_branch_cond_0 ? 4'b1110 : 4'b1111)
    // Slot-1-triggered conditions: the group survives (slot 0 must commit), so
    // aborting conditions clear one bit fewer than their slot-0 counterparts.
    & ($replay_cond_1     ? 4'b1110 : 4'b1111)
    & ($jump_cond_1       ? 4'b1100 : 4'b1111)
    & ($mispred_branch_cond_1 ? 4'b1100 : 4'b1111)
    & ($indirect_jump_cond_1 ? 4'b1100 : 4'b1111)
    & ($non_pipelined_cond_1 ? 4'b1000 : 4'b1111)
    & ($aborting_trap_cond_1 ? 4'b1000 : 4'b1111)
    & ($non_aborting_trap_cond_1 ? 4'b1000 : 4'b1111)
    // A predicted-taken branch is NON-aborting (the branch itself commits), so
    // for either slot it clears only the younger group -- same term as slot 0.
    & ($pred_taken_branch_cond_1 ? 4'b1110 : 4'b1111),
    1'b1}; // Shift in 1'b1 (fetch-valid).
$GoodPathMask[3+1:0] <=
  <<1$reset ? 5'b0 : // All bad-path (through self) on reset (next mask based on next reset).
  $next_good_path_mask;
```

The aborting-trap pair provides the clearest contrast: slot 0 uses $4'b0000$ (its own group and everything younger dies), whereas slot 1 uses $4'b1000$ (the group's own bit survives). That one-bit asymmetry constitutes the entire “slot 0 survives even when slot 1 aborts” guarantee. The predicted-taken terms are the one pair that is *identical* for both slots, for the reason given in Section 4.13.

4.7 Redirect Summaries and the Slot-1 Kill

Two summary signals answer the question “has the older instruction already diverted the PC?”—the first collecting slot 0's early redirect causes (known by stage 2), the second completing the set at stage 3:

```
$slot0_redir_e2 = /instr[0]$second_issue || /instr[0]$replay || /instr[0]$jump || /instr[0]
  ↳ $indirect_jump || /instr[0]$non_pipelined || /instr[0]$pred_taken_branch;
```



```
// Bypass-source availability, shared by both consuming slots.
$bypass_avail1_s0 = /instr[0]>>1$commit_dest_reg && ($GoodPathMask[1] || /instr[0]>>1
  ↳ $commit_second_issue);
$bypass_avail1_s1 = /instr[1]>>1$commit_dest_reg && ($GoodPathMask[1] || /instr[1]>>1
  ↳ $commit_second_issue);
$bypass_avail2_s0 = /instr[0]>>2$commit_dest_reg && ($GoodPathMask[2] || /instr[0]>>2
  ↳ $commit_second_issue);
$bypass_avail2_s1 = /instr[1]>>2$commit_dest_reg && ($GoodPathMask[2] || /instr[1]>>2
  ↳ $commit_second_issue);
$bypass_avail3_s0 = /instr[0]>>3$commit_dest_reg && ($GoodPathMask[3] || /instr[0]>>3
  ↳ $commit_second_issue);
$bypass_avail3_s1 = /instr[1]>>3$commit_dest_reg && ($GoodPathMask[3] || /instr[1]>>3
  ↳ $commit_second_issue);
@3
$slot0_redir_e3 = $slot0_redir_e2 || /instr[0]$mispred_branch || /instr[0]$aborting_trap || /
  ↳ instr[0]$non_aborting_trap;
```

Note the final term of `$slot0_redir_e2`: `$pred_taken_branch` was added when the two-bit predictor was ported. It is a stage-2 redirect, so it belongs in the stage-2 summary, and without it a predicted-taken branch in slot 0 would leave its own slot-1 fall-through instruction alive.

When slot 0 takes a *non-aborting* redirect—a jump, a predicted-taken branch, a resolved-taken branch, a non-pipelined bubble—the group is deliberately left good-path so slot 0 itself can commit. Slot 1, however, sits at the same mask bit and would wrongly commit as well. A dedicated kill signal removes exactly slot 1 in that situation and is folded into the commit decision:

```
$abort = 1'b0 || $no_fetch || $second_issue || $replay || $aborting_trap;

$killed_by_slot0 = (#instr != 0) && |fetch$slot0_redir_e3;
$good_path = (! $reset && |fetch>>-2$next_good_path_mask[3] && ! $killed_by_slot0);
$commit = $good_path && ! $abort;

// Conditions that commit results.
$commit_dest_reg = ($dest_reg_valid && $commit) || ($commit_second_issue );

$valid_ld = $ld && $commit;
$valid_st = $st && $commit;
```

4.8 The Six-Way Bypass Network

Where the baseline WARP-V forwarded from an instruction one, two, or three cycles ahead, three candidates, each of those cycles now holds two instructions, giving **six** possible cases. Six availability qualifiers are computed once in the shared scope and re-used by both consuming slots:

```
$bypass_avail1_s0 = /instr[0]>>1$commit_dest_reg && ($GoodPathMask[1] || /instr[0]>>1
  ↳ $commit_second_issue);
$bypass_avail1_s1 = /instr[1]>>1$commit_dest_reg && ($GoodPathMask[1] || /instr[1]>>1
  ↳ $commit_second_issue);
$bypass_avail2_s0 = /instr[0]>>2$commit_dest_reg && ($GoodPathMask[2] || /instr[0]>>2
  ↳ $commit_second_issue);
$bypass_avail2_s1 = /instr[1]>>2$commit_dest_reg && ($GoodPathMask[2] || /instr[1]>>2
  ↳ $commit_second_issue);
$bypass_avail3_s0 = /instr[0]>>3$commit_dest_reg && ($GoodPathMask[3] || /instr[0]>>3
  ↳ $commit_second_issue);
$bypass_avail3_s1 = /instr[1]>>3$commit_dest_reg && ($GoodPathMask[3] || /instr[1]>>3
  ↳ $commit_second_issue);
```


Operand read then selects among the six in **youngest-first** order; should two older instructions both have written the required register, the more recent write in program order must win:

```
/src[2:1]
$is_reg_condition = $is_reg && /instr$valid_decode;
? $is_reg_condition
  $rf_value[31:0] =
    |fetch/regs[$reg]>>3$value;
  {$reg_value[31:0], $pending} =
    ($reg == 5'b0) ? {32'b0, 1'b0} : // Read r0 as 0 (not pending).
    (|fetch$bypass_avail1_s1 && (|fetch/instr[1]>>1$wr_reg == $reg)) ? {|fetch/instr[1]>>1
    ↪ $rslt, |fetch/instr[1]>>1$reg_wr_pending} :
    (|fetch$bypass_avail1_s0 && (|fetch/instr[0]>>1$wr_reg == $reg)) ? {|fetch/instr[0]>>1
    ↪ $rslt, |fetch/instr[0]>>1$reg_wr_pending} :
    (|fetch$bypass_avail2_s1 && (|fetch/instr[1]>>2$wr_reg == $reg)) ? {|fetch/instr[1]>>2
    ↪ $rslt, |fetch/instr[1]>>2$reg_wr_pending} :
    (|fetch$bypass_avail2_s0 && (|fetch/instr[0]>>2$wr_reg == $reg)) ? {|fetch/instr[0]>>2
    ↪ $rslt, |fetch/instr[0]>>2$reg_wr_pending} :
    (|fetch$bypass_avail3_s1 && (|fetch/instr[1]>>3$wr_reg == $reg)) ? {|fetch/instr[1]>>3
    ↪ $rslt, |fetch/instr[1]>>3$reg_wr_pending} :
    (|fetch$bypass_avail3_s0 && (|fetch/instr[0]>>3$wr_reg == $reg)) ? {|fetch/instr[0]>>3
    ↪ $rslt, |fetch/instr[0]>>3$reg_wr_pending} :
    {$rf_value, |fetch/regs[$reg]>>3$pending};
// Replay if source register is pending.
```

Both the value and its pending bit are forwarded, exactly as in the baseline, and the identical six-way pattern is applied to the destination-pending check that keeps writes ordered.

4.9 Register File with Two Write Ports

The register file resided inside `/instr`; replicated unchanged, two register files would result. It was therefore hoisted into the shared scope and, since two instructions may now retire per cycle, equipped with **two write ports**:

```
/regs[31:1]
$wr0 = |fetch/instr[0]$commit_dest_reg && (|fetch/instr[0]$wr_reg == #regs);
$wr1 = |fetch/instr[1]$commit_dest_reg && (|fetch/instr[1]$wr_reg == #regs);
<<1$value[31:0] = $wr1 ? |fetch/instr[1]$rslt :
  $wr0 ? |fetch/instr[0]$rslt :
  $value;
// Write $pending along with $value, but coded differently because it must be reset.
<<1$pending = ! |fetch$reset && ($wr1 ? |fetch/instr[1]$reg_wr_pending :
  $wr0 ? |fetch/instr[0]$reg_wr_pending :
  $pending);
```

Both ports are explicit: `$wr0` for slot 0, `$wr1` for slot 1. The value multiplexer tests `$wr1` first, so a same-register, same-cycle conflict resolves in favour of **slot 1**, the younger instruction, whose write must persist. The write-after-write hazard of Section 5.3 prevents that exact collision from arising, but the priority is retained as a safety net. The four read ports (two sources for each of two slots) are served through the shared `/regs` references in the operand-read logic.

4.10 Shared Single-Port Data Memory

The data memory was hoisted for the same reason. Because the memory hazard of Section 5.4 guarantees at most one memory operation per group, a two-to-one multiplexer suffices to select the slot that drives the single port:

```
@3
$dm_slot1 = ! /instr[0]$ld_st_cond;    // The group's memory op (if any) is in slot 1.
$dm_spec_ld = /instr[0]$spec_ld || /instr[1]$spec_ld;
$dm_addr[31:0] = $dm_slot1 ? /instr[1]$addr : /instr[0]$addr;
$dm_valid_st = /instr[0]$valid_st || /instr[1]$valid_st;
$dm_st_value[31:0] = $dm_slot1 ? /instr[1]$st_value : /instr[0]$st_value;
$dm_st_mask[3:0] = $dm_slot1 ? /instr[1]$st_mask : /instr[0]$st_mask;
```

The bank-and-byte-mask store structure beneath the multiplexer is untouched from the baseline. The multiplexer is correct precisely *because* of the hazard rule: without it, both slots could contend for the port within one cycle.

4.11 Superscalar-Aware CSRs

The CSR counter state was hoisted so that a single copy is shared; software CSR writes are bound to slot 0 (enforced by the CSR hazard of Section 5.5). The retired-instruction counter was additionally made superscalar-aware: rather than incrementing by one per cycle, it accumulates the commit signals of *both* slots:

```
$full_csr_instret_hw_wr_value[63:0] = {$csr_instreth, $csr_instret} + {63'b0, /instr[0]$commit} +
    ↳ {63'b0, /instr[1]$commit}; // instret counts BOTH slots' commits.
```

Built-in IPC measurement

With `instret` counting both slots and `cycle` counting elapsed cycles, software can read the two counters and compute the core's live IPC on itself.

4.12 Load-Return

A returning load is a very old instruction and must never occupy the younger slot beside a newly-fetched instruction; program order would invert. Every load return is therefore steered into slot 0, and the slot from which the load originally issued is recorded. Because that slot may have been either one, the original-load record cannot be recovered with the baseline's version; it is instead reconstructed with explicit multiplexers keyed on the recorded slot:

```
/orig_load_inst
$dest_reg[4:0] = /instr$ld_ret_s1 ? |fetch/instr[1]>>5$dest_reg : |fetch/instr[0]>>5
↳ $dest_reg;
$spec_ld = /instr$ld_ret_s1 ? |fetch/instr[1]>>5$spec_ld : |fetch/instr[0]>>5
↳ $spec_ld;
$ld_st_word = /instr$ld_ret_s1 ? |fetch/instr[1]>>5$ld_st_word : |fetch/instr[0]>>5
↳ $ld_st_word;
$ld_st_half = /instr$ld_ret_s1 ? |fetch/instr[1]>>5$ld_st_half : |fetch/instr[0]>>5
↳ $ld_st_half;
$addr[31:0] = /instr$ld_ret_s1 ? |fetch/instr[1]>>5$addr : |fetch/instr[0]>>5
↳ $addr;
$raw_funct3[2:0] = /instr$ld_ret_s1 ? |fetch/instr[1]>>5$raw_funct3 : |fetch/instr[0]>>5
↳ $raw_funct3;
// Load data comes from the shared data memory (read at the load's stage 5).
$ld_data[31:0] = |fetch>>5$dm_ld_data;
```

The load result is then formatted with byte selection and sign extension as in the baseline, and the register-write destination selects the original load's destination for a return (the normal decoded destination otherwise). Every load result lands in the correct register.

4.13 Porting the Two-Bit Saturating Branch Predictor

The predictor described in Section 2.8 required four coordinated changes to become dual-issue-correct. The guiding rule is the same one that governs the register file and the data memory: *the predictor is shared machine state, so exactly one copy must exist, and it must observe every committed branch in program order.*

(1) Hoisting the counter and applying two updates per cycle

Left inside `/instr[1:0]`, the counter would have been replicated into two independent predictors, each seeing only half the branch history—architecturally harmless but functionally useless. It is therefore hoisted into `|fetch`.

Hoisting immediately raises a question the single-issue core never faced: both slots may retire a branch in the same cycle, so the counter may need *two* updates at once. Three options were considered:

Option	Assessment
Add a structural hazard and replay slot 1 whenever both slots hold a branch	Consistent with the memory and CSR ports, but branches are frequent, so the IPC cost would be real and recurring.
Let one slot win and drop the other update	Cheapest, but the counter would drift from the true branch history and predict less accurately.
Apply both updates in series within the cycle	Reproduces the baseline’s semantics exactly, costs no issue slots, and is almost free in area because the counter is only two bits wide. Chosen.

The decisive observation is that the predictor is *not* architectural state. A mis-updated counter costs performance, never correctness, so the structural hazard used for the memory and CSR ports would buy nothing here. Chaining two two-bit saturating steps is a trivial amount of logic and keeps the prediction history exact:

```
// TWO-BIT SATURATING BRANCH PREDICTOR (hoisted from /instr to |fetch).
// One counter serves the whole core, exactly as in the single-issue
// baseline. Both slots can commit a branch in the same cycle, so the
// two saturating steps are applied IN SERIES -- slot 0 (older) first,
// then slot 1 -- which reproduces the baseline's "one update per
// committed branch, in program order" semantics.
// 2'b00 strongly not-taken | 2'b01 weakly not-taken
// 2'b10 weakly taken      | 2'b11 strongly taken
$bp_upd_0 = /instr[0]$branch && /instr[0]$commit; // Slot 0 retires a branch.
$bp_upd_1 = /instr[1]$branch && /instr[1]$commit; // Slot 1 retires a branch.
$branch_or_reset = $bp_upd_0 || $bp_upd_1 || $reset;
// Step 1: apply the older slot's outcome to the current state.
$bp_state_0[1:0] =
    ! $bp_upd_0      ? $BranchState :
    /instr[0]$taken ? (($BranchState == 2'b11) ? 2'b11 : $BranchState + 2'b1) :
                  (($BranchState == 2'b00) ? 2'b00 : $BranchState - 2'b1);
// Step 2: apply the younger slot's outcome to the result of step 1.
$bp_state_1[1:0] =
    ! $bp_upd_1      ? $bp_state_0 :
    /instr[1]$taken ? (($bp_state_0 == 2'b11) ? 2'b11 : $bp_state_0 + 2'b1) :
                  (($bp_state_0 == 2'b00) ? 2'b00 : $bp_state_0 - 2'b1);
? $branch_or_reset
  $BranchState[1:0] <=
    $reset ? 2'b01 : // Weakly not-taken out of reset.
    $bp_state_1;
```

Slot 0's step is applied first and slot 1's step operates on its result, so the counter ends the cycle

in exactly the state a single-issue core would have reached after executing the same two branches consecutively. Both update terms are gated by `$commit`, so wrong-path branches still never disturb the counter, and `$RETAIN` is replaced by an explicit saturation constant because the intermediate value `$bp_state_0` is combinational rather than state.

Equivalence

For all four counter states and all sixteen combinations of slot activity and branch outcome, the chained update produces the same result as applying the same retirements one at a time in program order. The two-step formulation is therefore an exact reproduction of the baseline predictor’s behaviour, not an approximation.

(2) Per-slot prediction from the shared counter

The read side stays inside the replicated scope, since each slot must decide for its own instruction, but it reads the one shared counter:

```
// The saturating counter itself is shared machine state and lives in the
// |fetch scope. Each slot reads that one counter to form its own
// prediction, so both slots of a group predict from the same history.
@2
? $branch
    $pred_taken = |fetch>>2$BranchState[1]; // MSB of the 2-bit counter.
@2
// Redirect fetch to the branch target when the predictor says taken.
$pred_taken_branch = $pred_taken && $branch;
```

The `>>2` alignment of the baseline is preserved exactly; only the scope prefix changes, because the counter now lives in `|fetch`. Since a fetch group advances the pipeline by one cycle, this alignment now reaches two *groups* back rather than two instructions back—an inherent property of a wider machine, and the reason both slots of a group necessarily share one prediction.

(3) Splitting the predicted-taken redirect

The new stage-2 redirect is treated exactly like every other redirect condition: split per slot, qualified by the good-path mask, and—for slot 1—additionally qualified by the slot-0 summary so that a dead slot-1 branch cannot steer the PC:

```
// Two-bit predictor: a predicted-taken branch redirects fetch to the branch
// target at stage 2 (one stage earlier than a mispredict, which resolves at 3).
$pred_taken_branch_cond_0 = (/instr[0]>>1$pred_taken_branch && !(1'b0 || /instr[0]>>1$no_fetch
    ↪ || /instr[0]>>1$second_issue || /instr[0]>>1$replay) && $GoodPathMask[1]);
$pred_taken_branch_cond_1 = (/instr[1]>>1$pred_taken_branch && !(1'b0 || /instr[1]>>1$no_fetch
    ↪ || /instr[1]>>1$second_issue || /instr[1]>>1$replay) && (! >>1$slot0_redir_e2) &&
    ↪ $GoodPathMask[1]);
```

Its good-path mask term is `4'b1110` for *both* slots. This is the only redirect pair in the design that is symmetric, and the reason is instructive. A predicted-taken branch is **non-aborting**: the branch instruction itself is perfectly valid and commits normally. Neither slot’s own group may therefore be killed, so each clears only the single younger group. Contrast the replay pair, where slot 0 uses `4'b1100` because a replaying instruction must kill itself, while slot 1 uses `4'b1110` so that slot 0 survives.

Killing slot 1 of the *same* group when slot 0 predicts taken is handled not by the mask—which cannot

distinguish the two slots—but by adding `$pred_taken_branch` to the stage-2 slot-0 summary, so that `$skilled_by_slot0` removes the fall-through instruction:

```
$slot0_redir_e2 = /instr[0]$second_issue || /instr[0]$replay || /instr[0]$jump || /instr[0]
    ↪ $indirect_jump || /instr[0]$non_pipelined || /instr[0]$pred_taken_branch;
```

This single term is easy to overlook and essential: without it, a predicted-taken branch in slot 0 would redirect the fetch stream while its own fall-through neighbour in slot 1 continued to execute and commit.

(4) Direction-aware mispredict recovery

Finally, the mispredict target must account for predictions in both directions:

```
$mispred_branch = $branch && ! ($conditional_branch && ($taken == $pred_taken));
?$valid_decode_branch
    $branch_redir_pc[31:0] =
        // With the two-bit predictor a mispredict can go either way: a branch
        // predicted taken but resolved not-taken must redirect to PC+4.
        (! $taken) ? $pc_inc :
            $branch_target;
```

Because `$pc_inc` is already a per-slot signal (Section 4.4), each slot recovers to its own fall-through address, so a mispredicted branch in slot 1 correctly resumes at its own `PC + 4` rather than slot 0's.

Summary of the port

Baseline element	Dual-issue treatment
<code>\$BranchState</code> counter	Hoisted to <code> fetch</code> ; one counter, two chained saturating update steps (slot 0 then slot 1).
<code>\$pred_taken</code>	Stays per slot; reads the shared counter via <code> fetch>>2\$BranchState[1]</code> .
<code>\$pred_taken_branch</code>	Stays per slot; additionally added to <code>\$slot0_redir_e2</code> so slot 0 kills its own slot-1 neighbour.
<code>\$pred_taken_branch_cond</code>	Split into <code>_0</code> and <code>_1</code> ; slot 1 gains the <code>\$slot0_redir_e2</code> qualifier.
Good-path mask term	<code>4'b1110</code> for both slots (non-aborting redirect).
Next-PC arbiter	Two entries added below the replays, each supplying its own slot's <code>\$branch_target</code> .
<code>\$branch_redir_pc</code>	Direction-aware; uses the per-slot <code>\$pc_inc</code> .

No new intra-group hazard was introduced. The predictor is the one shared structure in the design that both slots may update in the same cycle without contending, which is what makes it different from the memory and CSR ports.

5 The Hazards of Dual Issue

Issuing two instructions per cycle creates conflicts that a single-issue machine never encounters. Four new data and structural hazards arise, and all four are resolved by one and the same mechanism.

5.1 Same-Cycle Conflicts

In a single-issue pipeline, every hazard is *between* instructions in *different* stages, covered by the forwarding network and the pending/replay mechanism. Dual issue adds a new category: two instructions in the **same stage, in the same cycle**. Slot 0 and slot 1 decode, read registers, and execute side by side; if they depend on one another or contend for a single-ported resource, the existing machinery cannot see it—that machinery only ever looks backward in time, never sideways within a cycle. These **intra-group hazards** are all detected at the operand-read stage:

```
$raw_haz = |fetch/instr[0]$dest_reg_valid &&
           ((/src[1]$is_reg_condition && (/src[1]$reg == |fetch/instr[0]$dest_reg)) ||
           (/src[2]$is_reg_condition && (/src[2]$reg == |fetch/instr[0]$dest_reg))); // Read-
           ↳ after-write.
$waw_haz = |fetch/instr[0]$dest_reg_valid && $dest_reg_valid && ($dest_reg == |fetch/instr[0]
           ↳ $dest_reg); // Write-after-write (needed when slot 0 is a load; kept for all cases for
           ↳ simplicity).
$mem_haz = (|fetch/instr[0]$ld_st && |fetch/instr[0]$valid_decode) && ($ld_st && $valid_decode);
           ↳ // Structural: one data-memory port.
$csr_haz = $is_csr_instr && $valid_decode; // Structural: CSR instructions execute in slot 0
           ↳ only.
$intra_group_replay = (#instr != 0) && $valid_decode &&
                    ($raw_haz || $waw_haz || $mem_haz || $csr_haz);

// Combined replay for this slot.
$replay = ($pending_replay ) || $intra_group_replay;
```

The four detector conditions—`$raw_haz`, `$waw_haz`, `$mem_haz`, `$csr_haz`—are examined below. Note that the branch predictor deliberately contributes *no* fifth detector: as Section 4.13 explains, its counter accepts both slots’ updates in the same cycle, so it is the one shared structure that never needs a structural hazard.

5.2 RAW Read-After-Write Between the Slots

Nature. Slot 1 reads a register that slot 0 writes in the very same cycle. Slot 1 requires slot 0’s result, but the two instructions execute side by side and the result does not yet exist. The detector asserts when slot 0 holds a valid destination and either of slot 1’s decoded source indices matches it.

Why in-cycle forwarding was rejected. Forwarding slot 0’s result into slot 1 within the cycle would chain the two ALUs into one long combinational path, roughly doubling the execute-stage delay and degrading the achievable clock frequency. The cost is a few bubble cycles on slot 1, incurred only when a true dependency exists.

Resolution. Replay slot 1 (Section 5.6).

5.3 WAW Write-After-Write Between the Slots

Nature. Both slots write the same destination register in the same cycle; the writes must not land out of program order.

Role of loads. The check is strictly required only when slot 0 is a *load*: the load’s later second-issue return would otherwise overwrite slot 1’s newer value with stale data.

Resolution. Replay slot 1 (Section 5.6).

5.4 STRUCTURAL Contention on the Memory Port

Nature. The data memory exposes one load/store port, yet a fetch group may contain two memory instructions, which cannot both access the port in one cycle.

Resolution. Replay slot 1 (Section 5.6). The fix yields a bonus: it *guarantees* that at most one memory operation issues per group, which is precisely the property that renders the single-port memory multiplexer of Section 4.10 correct.

5.5 STRUCTURAL Contention on the CSR Port

Nature. The CSR counter state exposes one software-write port, wired to slot 0; a CSR instruction may therefore execute only in slot 0.

Resolution. Replay slot 1 (Section 5.6). A CSR instruction landing in slot 1 re-executes as slot 0 of a later group, where it holds legitimate access to the write port.

5.6 The Replay Resolution

All four hazards are resolved **identically**. The detectors are combined into a single condition, gated to slot 1 alone, and folded into the existing replay signal:

```
$intra_group_replay = (#instr != 0) && $valid_decode &&
                    ($raw_haz || $waw_haz || $mem_haz || $csr_haz);

$replay = ($pending_replay) || $intra_group_replay;
```

Effect of a slot-1 replay

A replay steers the PC to **slot 1's own address**. The offending slot-1 instruction is discarded and **re-fetched as slot 0 of a slightly later group**. By the time it returns as slot 0, the instruction it depended on (or the resource it required) has advanced far enough down the pipeline that the ordinary cross-group forwarding network supplies the value, or the shared port is free. Slot 0 of the original group is entirely unaffected and commits normally.

A hazard costs slot 1 its issue opportunity for a couple of cycles but **never stalls slot 0**. No dedicated stall logic, no ALU-to-ALU forwarding cascade, and no memory arbiter were required; every new hazard collapses into “slot 1 yields and retries as an older instruction.”

Hazard	Trigger	Resolution
RAW	Slot 1 reads slot 0's destination	Replay slot 1
WAW	Both slots write one register	Replay slot 1
Memory	Two memory ops in one group	Replay slot 1
CSR	CSR instruction in slot 1	Replay slot 1
Branch predictor	Two branches in one group	<i>None needed</i> — both updates applied in series

5.7 Control Hazards Between the Slots

Beyond the four data and structural hazards, dual issue complicates **control** hazards—the cases where a branch, jump, or trap in one slot must redirect the fetch and invalidate the other. These are handled not by replay but by the mask and kill logic of Section 4:

- **Slot 0 redirects.** Slot 1 of the same group is wrong-path. If the redirect is *aborting* (a trap), the mask clears the group's own bit and both slots die. If it is *non-aborting* (jump, predicted-taken

branch, mispredicted branch, non-pipelined bubble), the group remains good so slot 0 commits, and `$killed_by_slot0` removes slot 1 alone.

- **Slot 1 redirects.** Slot 0 is older and must survive. The slot-1 mask terms clear one bit fewer than their slot-0 counterparts, so the group's own bit stays set and slot 0 commits; slot 1's non-commit follows from its per-slot abort.
- **Both slots redirect at different stages.** The priority multiplexer (later stage wins; slot 0 before slot 1) together with the shifting mask sequences these correctly across cycles; the races have been discussed in Section 6.2.

A branch can now redirect twice

With the two-bit predictor a single branch may steer the program counter at stage 2 (predicted taken) and again at stage 3 (mispredicted). The two redirects never conflict: the stage-3 mispredict has strictly higher priority in the next-PC multiplexer and its mask term clears one bit more, so it cleanly overwrites the speculative fetch its own earlier prediction started.

6 Some Extra Analysis

6.1 Replay Timing

A replayed slot-1 instruction must find its operand on the second attempt, and the timing is exact. The replay condition is consumed one cycle deep, so the re-fetched instruction trails the original group by precisely two transactions. If the original slot 0 was itself a load, the forward carries a pending indication, and the ordinary pending/replay loop takes over until the load returns.

6.2 Cross-Slot Redirect Races

Two slots of one group can trigger conditions at different stages, resolving in different cycles. The delicate races were traced explicitly:

- **Slot 1 replays in cycle T ; slot 0 proves to be a mispredicted-taken branch in cycle $T+1$.** The replay redirects to slot 1's address, clearing only strictly-younger mask bits, so the group survives. In the next cycle the mispredict finds its mask bit still set, fires with higher (later-stage) priority, redirects to the branch target, and its mask term kills the replay-path fetch. Slot 0 commits; slot 1 is correctly dead (it was never on the taken path); fetch resumes at the target. The shifting mask performs the entire sequencing without any special case.
- **Slot 0 replays; slot 1 is a jump.** The slot-0 replay is aborting for the group and clears the group's own bit; the slot-1 jump is mask-suppressed in the following cycle, and the group re-fetches from slot 0.
- **Slot 0 is a jump; slot 1 would trap one stage later.** The group remains good-path (jumps are non-aborting), so the kill signal and the slot-1 redirect qualifier are what prevent the already-dead slot-1 trap from wrongly redirecting the PC after slot 0 has done so.
- **Slot 0 predicts taken at stage 2; slot 1 is also a branch.** `$pred_taken_branch` is part of `$slot0_redir_e2`, so slot 1 is killed by `$killed_by_slot0` and its own `$pred_taken_branch_cond_1` is suppressed by the same summary. Slot 1 therefore neither commits nor updates the predictor, which is correct: it was never on the taken path.

6.3 Predictor Behaviour Under Dual Issue

Two properties follow from sharing one counter between two slots and are worth stating explicitly.

- **Both slots of a group predict identically.** They read the same counter in the same cycle, so if a group contains two branches, both receive the same prediction. Only slot 0's prediction can take effect, because a predicted-taken branch in slot 0 kills slot 1 outright.
- **Predictor state remains exact.** Because the two update steps are chained in program order and both are gated on `$commit`, the counter after any cycle equals the value a single-issue core would hold after retiring the same sequence of branches. Widening the machine changes prediction *timing*, never predictor *state*.

A Summary of Changes

Area	Change
Instruction memory	Widened to a power-of-two depth with NOP padding so the slot-1 fetch is always in bounds.
Instruction slot	Per-instruction scope replicated into two slots—the superscalar mechanism itself.
Program counter	Per-slot PC (slot 0 at the group PC, slot 1 at PC+4); default stride widened to eight bytes.
Address computation	Branch, jump, AUIPC, and link addresses switched to each slot's own PC.
Next-PC	Every redirect split into slot-0 and slot-1 forms; priority multiplexer with later-stage-wins and slot-0-before-slot-1 ordering.
Good-path mask	Slot-1 aborts clear one fewer bit so slot 0 always survives.
Redirect summaries and kill	Slot-0 redirect summaries added, plus a kill signal that removes slot 1 alone.
Forwarding network	Extended from three to six candidates (both slots at each depth), youngest first.
Register file	Hoisted to the shared scope; two write ports, slot 1 winning ties.
Data memory	Hoisted to the shared scope; single port fed by a two-to-one slot multiplexer.
CSR counters	State hoisted and shared; software writes bound to slot 0; <code>instret</code> counting both slots.
Loads	Returns routed through slot 0; original-load record rebuilt with explicit multiplexers.
Branch predictor state	<code>\$BranchState</code> hoisted to the shared scope; two saturating update steps chained in program order so both slots may retire a branch in the same cycle.
Branch prediction read	Kept per slot, reading the one shared counter through <code> fetch>>2\$BranchState[1]</code> .
Predicted-taken redirect	Split into slot-0 and slot-1 conditions with a <code>4'b1110</code> mask term for each; two entries added to the next-PC multiplexer; <code>\$pred_taken_branch</code> added to <code>\$slot0_redir_e2</code> .
Mispredict recovery	<code>\$branch_redir_pc</code> made direction-aware, recovering to the per-slot <code>\$pc_inc</code> when a predicted-taken branch resolves not-taken.
Hazard logic	RAW, WAW, memory, and CSR intra-group hazards detected and resolved by replaying slot 1.